

More Probability

Math 252

1. **Bob's Boxes.** Bob has two beautiful boxes of balls. The first contains 2 green balls and 7 red balls. The second contains 4 green balls and 3 red balls. Bob flips a coin to randomly select a box. Then he randomly selects one ball from that box. If the selected ball is red, what is the probability that it was chosen from the first box?

Let's build up to this step by step.

- a. Is every ball equally likely to be selected? If not, which are more likely and which less likely?
- b. Let's define some events.
 - A : Bob selects box 1
 - B : Bob selects box 2
 - R : Bob selects a red ball
- c. Let's do some inventory. Which of these probabilities can we easily determine? Which is our main question? How can you use the items on the list that you know to figure out the values you don't know? (Feel free to add additional probabilities to the inventory if that is helpful.)

$$\begin{array}{ccccccc}
 p(A) & p(B) & p(R) & & & & \\
 p(A \cap B) & p(A \cup B) & p(A \cap R) & p(B \cap R) & & & \\
 p(A | R) & p(R | A) & p(B | R) & p(R | B) & p(R | A) & &
 \end{array}$$

2. **Random Primality Testing.** Miller's test for primality of a number n uses arithmetic mod n and a base b with $1 < b < n$. For a given b , the test either correctly determines that n is composite or it is inconclusive. It can be shown that for any composite number, the test is conclusive (and correct) for at least $3/4$ of the possible values of b . For a prime number, the test is always inconclusive. This leads to an interesting randomized test:

```

string PrimeTest(n, attempts) {
  for (i = 0; i < attempts; i++) {
    select b at random with 1 < b < n
    if (MillersTest(n, b) == "composite") {
      return("composite (for sure)")
    }
  }
  return("prime (well, probably)")
}

```

To make life easier, let's assume the probability that a composite number yields a conclusive test is exactly $3/4$.

- a. If n is prime, what is the probability that `PrimeTest(n, 5)` returns "prime (well, probably)"?
- b. If n is composite, what is the probability that `PrimeTest(n, 5)` returns "prime (well, probably)"?
- c. If n is composite, what is the probability that `PrimeTest(n, 10)` returns "prime (well, probably)"?
- d. If n is composite, what is the probability that `PrimeTest(n, 20)` returns "prime (well, probably)"?

Note: For RSA purposes, once we get two large probably prime numbers, we can try encoding and decoding a message. If a composite number sneaks through the randomized test, decoding won't work and we can try again with a different pair of probably prime numbers.

3. **Medical Test.** A test for a rare medical condition that affects 1 person in 10,000 has the following properties: If a person is healthy, it correctly diagnoses this 98% of the time. If a person is diseased, it correctly diagnoses this 99% of the time. If you take the test and it comes back positive (ie, the test says you have the disease), what is the probability that you have the disease?

Hint: Use the inventory method. Useful events include

- H : person is healthy
- D : person is diseased
- P : test is positive (indicates disease)
- N : test is negative (indicates healthy)

4. **Hashing.** A hash function h assigns one of m storage locations (an index in an array, for example) to each key k . Typically there are many more keys than storage locations, but only a much smaller number of keys will occur in a particular application. For example, if the keys are social security numbers, then there are approximately 350 active social security numbers (keys), but a give application may have data on only a very small fraction of these people. If $h(k_1) = h(k_2)$, then we say there is a collision (because we would like to put both keys into the same storage location and have to figure out some work-around).

Good hash functions have the property that if a key is selected at random, then the probability of being mapped to a specified storage location is approximately $1/m$, where m is the number of storage locations. (So each storage location is roughly equally likely to be selected.) For the questions below, assume that the probability is exactly $1/m$. That should be a good approximation for good hash functions.

- a. If a hash function is mapping to 20 storage locations, what is the probability that we can place 4 keys without any colisions? (That is, what is the probability that the first four hash values will all be different.)
- b. If a hash function is mapping to 50 storage locations, what is the probability that we can place 10 keys without any colisions? (That is, what is the probability that the first ten hash values will all be different.)
- c. If a hash function is mapping to 300 storage locations, what is the probability that we can place 25 keys without any colisions?
- d. Make up a similar problem of your own with a different number of storage locations and a different number of keys to place.
- e. This is probably too big for your calculator, but if m is 1 million, then the probability of placing 1178 keys without any collisions is $\approx 1/2$.
- f. Explain why these results indicate that hash tables will need to deal with colisions if they are going to be memory efficient.